Guía de Resolución: Tienda de Música

DWEC · MÉTODOS DE ARRAYS Y PARADIGMA FUNCIONAL EN JAVASCRIPT

1. Glosario de Métodos Utilizados

Herramientas clave de JavaScript empleadas en la solución de los ejercicios:

`Array.isArray(x)` : Comprueba si un valor es un array. Devuelve `true` o `false`.

`[...array]` : Operador Spread. Copia elementos dentro de un nuevo array sin mutar el original.

`.sort(fn)` : Reordena un array. Usamos `a - b` para números y `localeCompare` para texto.

`.map(fn)` : Recorre un array, aplica una función a cada elemento y devuelve un **nuevo array**.

`[a, b]` : Desestructuración. Extrae valores de un array directamente a variables individuales.

`.slice(a, b)` : Corta una porción del array desde el índice `a` hasta el `b` (no incluido).

2. Paso a Paso de los Ejercicios

Ejercicio 1.1 — crearCatalogo(matriz)

Transformación

Objetivo: Convertir una matriz de sub-arrays `[nombre, categoria, precio, stock]` en un array de objetos.

1. **Validación:** Si el parámetro no es un array o está vacío, devuelve un array vacío `[]`.
2. **Mapeo y Desestructuración:** Recorre cada fila con `.map()` desglosando las variables directamente.
3. **Retorno:** Devuelve el objeto delimitado entre paréntesis `({ ... })` para retorno explícito.

```
export const crearCatalogo = (matriz) => {
  if (!Array.isArray(matriz) || matriz.length === 0) {
    return [];
  }
  return matriz.map(([nombre, categoria, precio, stock]) => ({
    nombre,
    categoria,
    precio,
    stock
  }));
};
```

Ejercicio 1.2 — ampliarCatalogo(catalogo, matrizNovedades)

Inmutabilidad

Objetivo: Añadir las novedades al catálogo sin modificar el array original.

1. **Protección de datos:** Asigna arrays vacíos por defecto si los argumentos de entrada son inválidos.
2. **Reutilización:** Convierte la `matrizNovedades` en objetos usando la función `crearCatalogo()`.
3. **Combinación limpia:** Devuelve un nuevo array utilizando el operador spread `[...cat, ...catalogoNovedades]`.

```
export const ampliarCatalogo = (catalogo, matrizNovedades) => {
  const cat = Array.isArray(catalogo) ? catalogo : [];
  const novedades = Array.isArray(matrizNovedades) ? matrizNovedades : [];

  const catalogoNovedades = crearCatalogo(novedades);
  return [...cat, ...catalogoNovedades];
};
```

Ejercicio 1.3 — nombresOrdenados(catalogo)

Ordenación de texto

Objetivo: Extraer nombres y ordenarlos alfabéticamente respetando tildes y caracteres en español.

1. **Extraer:** Mapea el catálogo para obtener una lista simple únicamente con `producto.nombre`.
2. **Ordenar tildes:** Utiliza a `.localeCompare(b, 'es')` dentro de `.sort()` para un orden lingüístico correcto.

```
export const nombresOrdenados = (catalogo) => {
  if (!Array.isArray(catalogo) || catalogo.length === 0) return [];

  const nombres = catalogo.map((producto) => producto.nombre);
  return nombres.sort((a, b) => a.localeCompare(b, 'es', { sensitivity: 'base' }));
};
```

Ejercicio 1.4 — ordenarPorPrecio(catalogo, descendente)

Copia y Ordenación

Objetivo: Devolver una copia del catálogo ordenada por precio (ascendente o descendente).

1. **Copia pura:** Duplica el array con `[...catalogo]` porque `.sort()` muta el array sobre el que actúa.
2. **Comparación numérica:** Usa `a.precio - b.precio` (ascendente) o `b.precio - a.precio` (descendente).

```
export const ordenarPorPrecio = (catalogo, descendente = false) => {
  if (!Array.isArray(catalogo)) return [];

  const copiaCatalogo = [...catalogo];
  return copiaCatalogo.sort((a, b) => {
    return descendente ? b.precio - a.precio : a.precio - b.precio;
  });
};
```

Ejercicio 1.5 — tresMasBaratos(catalogo)

Composición de funciones

Objetivo: Obtener los nombres de los 3 productos con los precios más bajos del catálogo.

1. **Reutilizar:** Llama a `ordenarPorPrecio(catalogo, false)` para obtener la lista ordenada de menor a mayor.
2. **Recortar:** Aplica `.slice(0, 3)` para extraer solo los primeros 3 elementos.
3. **Proyectar:** Transforma la selección con `.map(p => p.nombre)` para devolver únicamente sus nombres.

```
export const tresMasBaratos = (catalogo) => {
  if (!Array.isArray(catalogo) || catalogo.length === 0) return [];

  const ordenados = ordenarPorPrecio(catalogo, false);
  return ordenados.slice(0, 3).map((producto) => producto.nombre);
};
```

3. Resumen de Buenas Prácticas

✔ **Evita mutaciones:** No uses `.push()` o `.sort()` directamente sobre el catálogo original; utiliza siempre copias con el operador spread (`[...]`).
✔ **Defensa contra errores:** Valida siempre si lo recibido es un array mediante `Array.isArray()` antes de ejecutar operaciones de array.
✔ **Reutilización de código:** Apoyarse en funciones previamente creadas (como `crearCatalogo` u `ordenarPorPrecio`) reduce duplicidades y facilita el mantenimiento.